Parallel chess search: C++ implementation and Vanda experiments

Additional CPU measurements on six chess positions (24 full-node scaling configurations, three trials each) are in the multi-position archive. It includes raw logs, serial baselines, strategy comparisons and regenerated summary tables.

This project studies fixed-depth negamax with alpha-beta pruning. The current program provides an exhaustive minimax validation mode, a serial CPU reference, root- and second/third-level OpenMP CPU versions, a seeded-root variant that searches the first move before parallelizing the others, optional MPI + OpenMP versions, and two OpenMP target versions of alpha-beta search over legal trees generated on the CPU. All versions use the same material evaluation and chess terminal rules. Vanda CPU and GPU results, including repeated scaling runs, are in `experiments/`. The draft `report/main.pdf` discusses the algorithm and evidence. The main hybrid parallel implementation is MPI between nodes plus OpenMP threads on CPUs. OpenMP GPU target offload is an additional feasibility study.

Third-party code and ownership

`third_party/chess-library/chess.hpp` is the unmodified header from Disservin/chess-library, commit `2a608d58f44776909bb6e2c09505e97ba1451319`, licensed under MIT. Its licence is copied verbatim in `third_party/chess-library/LICENSE`. SHA-256 of the vendored header:

```
905b95c2ac870e488a5262c992e3aed0907a2233eb95b9a2835a3d47d9a3cf5d
third_party/chess-library/chess.hpp
```

The library supplies board state, FEN parsing, legal move generation, move application and reversal, draw-rule helpers, and UCI move formatting. Search, evaluation, CPU scheduling, GPU tree representation/traversal, instrumentation, and command-line code in `src/main.cpp` are project code. The library does not provide this program's search algorithm. The instructor approved using an externally credited legal-move library. The vendored library is used only on the CPU; none of its functions are called from an OpenMP target region. The GPU tree representation, traversal, and leaf evaluation are implemented in this project's source.

Build

CPU: C++17 compiler and OpenMP runtime.

```
make cpu
make test
```

MPI + OpenMP, if `mpic++` is available:

```
make mpi
mpirun -n 2 build/chess_search_mpi --backend mpi --threads 4 --depth 4
```

The local development environment has g++ but no `mpic++` or functioning GPU driver. MPI and GPU versions have compiled and run on Vanda. For the GPU binary, use an NVIDIA HPC SDK `nvc++` installation with OpenMP target support:

```
make gpu GPU_CXX=nvc++ GPU_FLAGS='-O2 -std=c++17 -mp=gpu'
OMP_TARGET_OFFLOAD=MANDATORY build/chess_search_gpu --backend gpu --depth 3
```

Inspect `ran_on_device=1` in the output. The program refuses `--backend gpu` if a target region ran on the host. `--backend gpu-host` explicitly permits host fallback for correctness checks without a GPU. Compiler flags and GPU availability must be verified on the allocated Vanda node.

The complete report integrates the original CPU/GPU measurements and the six-position CPU archive. See `report/main.pdf` and `report/README.md` for its scope, evidence and rebuild steps. The five vector figures and CPU result tables are regenerated from the archived measurements with `report/build_analysis.py`.

NUS Vanda batch jobs

Vanda uses **PBS**, so the scripts in `scripts/` use #PBS resource requests and are submitted with `qsub`, not Slurm. The templates request modest resources and inherit your loaded modules with `#PBS -V`. On the Vanda login node, inspect available compiler/MPI modules and load a compatible toolchain before submitting. The actual module names, queue, account/project, and node limits depend on your HPC account and must be checked there.

```
qsub scripts/vanda_compile.pbs
qsub scripts/vanda_cpu.pbs
qsub scripts/vanda_gpu.pbs
qstat
```

The compile-only job saves `logs/mpi-compile-$PBS_JOBID.log`. If `mpic++` is absent, first inspect `module avail` on Vanda and load the available compiler/MPI pair before submitting again. Do not guess a module name from another NUS cluster. Keep the complete compile log if asking for help: the first compiler error is usually more useful than the last line.

The module listing supplied for this account includes a matched CPU pair:

```
module load GCC/14.2.0 OpenMPI/5.0.7-GCC-14.2.0
command -v mpic++
mpic++ --version
qsub scripts/vanda_compile.pbs
```

For a separate GPU submission, the listing includes `NVHPC/24.7-CUDA-12.3.0`. Load it together with `GCC/13.2.0` in a fresh login shell, check `command -v nvc++`, then submit `scripts/vanda_gpu.pbs`. The script passes `--gcc-toolchain=$EBROOTGCCCORE` to `nvc++`; without this option, the first A40 job could not find the standard C++ header `<functional>`. Vanda A40 jobs 1499986, 1499992, and 1518429 confirmed `ran_on_device=1` at depths 3, 4, and 5, with the same score and move as serial. See `` `experiments/` `` for the measured overhead and the bounded GPU conclusion.

For on-demand chunks, set `JOB_GPU_BACKEND=gpu-chunk` and `JOB_GPU_CHUNK_DEPTH=2` or 3 when submitting `scripts/vanda_gpu.pbs`. `scripts/vanda_gpu_compare.pbs` repeats serial, original GPU, and both chunk depths three times on one A40 allocation. Its results are recorded in `experiments/raw/1518527-gpu-compare.txt`. `scripts/vanda_gpu_profile.pbs` repeats the explicit-transfer profiling variants at depth 5 and runs a depth-8 chunk profile. Its measurements are recorded in `experiments/raw/1518578-gpu-profile.txt`.

The depth-5 A40 pilot used `JOB_DEPTH=5` and `JOB_MAX_TREE_NODES=7000000` when submitting `scripts/vanda_gpu.pbs`. The latter increases the explicit CPU tree-build cap; it does not change the search result.

The CPU template requests one chunk with 4 CPU cores, 1 MPI rank and 4 OpenMP threads. To run multiple nodes, edit its `#PBS -l select=...` to request more chunks while keeping `ncpus >= mpiprocs * ompthreads` per chunk; set `JOB_THREADS` to match `ompthreads`. `mpirun` uses the rank count in `$PBS_NODEFILE`. The GPU template requests one GPU and checks for a real device. Both scripts save output under `results/$PBS_JOBID/`. They do not submit jobs automatically. Run one short allocation first and check the compiler, rank placement, GPU device detection, and serial score before expanding the matrix.

For a same-position depth-7 CPU pilot, the already verified `1 rank x 4 threads` configuration took 7.35 seconds on Vanda. To get a true serial alpha-beta baseline, the script now accepts `JOB_BACKEND=serial`; request one CPU and set its split depth to 1. PBS command-line resource requests override the template's defaults:

```
qsub -l select=1:ncpus=1:mpiprocs=1:ompthreads=1:mem=8gb \
  -v JOB_BACKEND=serial,JOB_THREADS=1,JOB_SPLIT_DEPTH=1,JOB_DEPTH=7 \
  scripts/vanda_cpu.pbs
```

For a two-node pilot with one rank and four threads per node, request `place=scatter` so each PBS chunk lands on a distinct host:

```
qsub -l select=2:ncpus=4:mpiprocs=1:ompthreads=4:mem=8gb \
  -l place=scatter -v JOB_DEPTH=7 scripts/vanda_cpu.pbs
```

Inspect the returned job's `Resource_List.select`, `Resource_List.place`, and the unique hostnames in `results/$PBS_JOBID/run.txt` before calling it a two-node result. Repeat each configuration after the pilot for

report data.

The CPU launcher uses `mpirun --bind-to none --report-bindings` by default so OpenMP threads can run on the cores PBS assigned to the job. Set `JOB_BINDING=default` to test Open MPI's default binding policy. Keep the PBS `ncpus` and `ompthreads` requests consistent with `JOB_THREADS`. The first depth-7 runs with default binding showed a 4-thread run slower than a 1-thread run; the binding report confirmed it was restricted to one core. Set `JOB_REPEATS=3` to run three sequential trials in one allocation. The output marks each trial and records SHA-256 hashes of the source and PBS script. Final figures should use medians across those trials.

`--backend omp-pv` and `--backend mpi-pv` perform a full-window search of the first legal root move, then use that score as a fixed bound for the other root moves. A move that cannot beat the bound may return an inexact score; the maximum root score remains exact. This preserves substantially more alpha-beta pruning while exposing only one task per remaining root move, so it is a useful contrast with the depth-2/3 frontier strategy. Both PV modes require `--split-depth 1`. On Vanda, set `JOB_BACKEND=mpi-pv,JOB_SPLIT_DEPTH=1` in the CPU PBS job. The CPU script accepts `JOB_FEN_PRESET=startpos|kiwipete|promotion|enpassant` and logs the full FEN for each job; an explicit `JOB_FEN` overrides the preset. This makes multi-position validation straightforward with `qsub -v`.

NUS's PBS resource guide documents `select`, `ncpus`, `mpiprocs`, and `ompthreads`; a Vanda course guide confirms that Vanda uses PBS. The scripts are templates until verified on the account's actual Vanda environment.

Upload and first build on Vanda

After the first upload, the project-local Git alias `git sync-hpc` runs `scripts/git-sync-hpc` from anywhere in this Git repository. Use `git sync-hpc --dry-run` to preview changes, then `git sync-hpc` to transfer them. It excludes local Git metadata, `build/`, `logs/`, and `results/`, and does not delete remote files. The default destination is `e1538331@vanda.nus.edu.sg:~/dsa5208/project3/`; set `CHESS_HPC_DEST` to override it. Git aliases live in the local repository configuration and are not included in the source submission; on a fresh clone, register it with `git config --local alias.sync-hpc '!project3/scripts/git-sync-hpc'`.

From this project's directory on your own computer, package only source and instructions (the generated `build/` directory is deliberately excluded):

```
tar -czf /tmp/dsa5208-project3-src.tar.gz Makefile README.md THIRD_PARTY.md src tests scripts
third_party
scp /tmp/dsa5208-project3-src.tar.gz NUSNETID@vanda.nus.edu.sg:~/
ssh NUSNETID@vanda.nus.edu.sg
```

Replace `NUSNETID` with your own ID. On Vanda, unpack and inspect the toolchain before submitting the compile job:

```
mkdir -p ~/dsa5208/project3
tar -xzf ~/dsa5208-project3-src.tar.gz -C ~/dsa5208/project3
cd ~/dsa5208/project3
module avail
command -v mpic++
hpc project
qstat -q
qsub scripts/vanda_compile.pbs
qstat -u "$USER"
```

If `mpic++` appears only after a module load, load that module and submit with the same environment (`#PBS -V` is already in the template). After the job ends, inspect `logs/mpi-compile-*.log` and the PBS output file `chess-build.o*`. The build itself runs on an allocated compute node. Once it succeeds, submit the CPU template, then the GPU template after checking that a GPU compiler module is available. Off-campus SSH access may require NUS VPN, according to the Vanda access guide.

`qsub` prints a job ID. Use `qstat -f JOB_ID` for its state and allocation, then `cat logs/mpi-compile-JOB_ID.log` for build diagnostics. From your own computer, retrieve the log with `scp NUSNETID@vanda.nus.edu.sg:~/dsa5208/project3/logs/mpi-compile-JOB_ID.log .`. If your account requires an explicit PBS project, use the project reported by `hpc project` with `qsub -P PROJECT scripts/vanda_compile.pbs` (and the same option for later jobs). Do not insert another course's project code.

GCC runtime mismatch seen in the first CPU job

The first Vanda MPI run (`1499851.stdct-mgmt-02`) compiled but exited before searching: the launched process picked `/lib64/libgomp.so.1` and `/lib64/libstdc++.so.6`, which lacked GCC 14 symbols. The updated Makefile embeds the selected GCC runtime directories in the MPI executable. The CPU PBS script also prepends those directories to `LD_LIBRARY_PATH`, forwards that variable through `mpirun -x`, and prints the `ldd` resolution into `results/$PBS_JOBID/run.txt`. Reload the matched GCC/OpenMPI modules, upload the updated Makefile and script, then submit a fresh CPU job. Check that `ldd` resolves both libraries from the GCC 14 module, and that a search result line follows. The `libpbs.so.0` component message in the earlier log was marked ignored; if launch still fails after fixing libraries, inspect the new PBS output for the next error.

Run

```
build/chess_search --backend serial --depth 4
build/chess_search --backend minimax --depth 3
build/chess_search --backend omp --threads 4 --depth 4
build/chess_search --backend omp --threads 4 --split-depth 2 --depth 4
build/chess_search --backend gpu-host --depth 3
build/chess_search --backend gpu-chunk-host --depth 5 --gpu-chunk-depth 3
build/chess_search --backend gpu-chunk-profile-host --depth 5 --gpu-chunk-depth 3
build/chess_search --backend serial --depth 3 \
  --fen 'r3k2r/p1ppqpb1/bn2pnp1/3PN3/1p2P3/2N2Q1p/PPPBBPPP/R3K2R w KQkq - 0 1'
```

The score is in centipawns from the side to move at the root. Mate scores are near +30000 or -30000. Equal-score moves are broken by the library's stable legal-move order. The program prints visited nodes, leaves, cutoffs, wall time, and, for target runs, tree size, CPU preparation time, target-region time, and whether a real device was used. `--max-tree-nodes` (default 2 million) limits GPU host-side expansion; the limit throws an error rather than silently dropping branches. Fixed depth is currently limited to 12 by the device traversal stack; useful GPU tree depths may be much lower due to complete host-side expansion.

`gpu-chunk` is an additional OpenMP target backend. CPU alpha-beta descends through the tree and builds only reached subtrees with at most `--gpu-chunk-depth` remaining plies. Each bounded subtree goes to the GPU, whose workers search its immediate child branches. This preserves pruning above the chunk but still builds every legal node within each reached chunk. `gpu-chunk-host` explicitly permits host fallback for validation. Output includes cumulative generated `tree_nodes` and `gpu_chunks` (chunk target launches, plus one separate device probe) to expose the batch-size tradeoff.

`gpu-profile` and `gpu-chunk-profile` use explicit OpenMP device allocations and copies to report `h2d_seconds`, `kernel_region_seconds`, and `d2h_seconds`, alongside an initial device probe and allocation time. The kernel-region timer includes launch and synchronization, so it does not isolate pure GPU arithmetic. The corresponding `*-host` modes allow local fallback checks. See the profiling results in `experiments/README.md`.

`--split-depth 1` assigns legal root moves as independent tasks. For `--backend omp` or `mpi`, values 2 or 3 generate more tasks from a deeper frontier and combine exact subtree scores by negamax. This can feed more CPU workers but may lose additional pruning. Both GPU backends reject larger `--split-depth` values; only `gpu-chunk` uses `--gpu-chunk-depth`.

What the original GPU version actually computes

The CPU uses the third-party library to generate the **complete legal game tree** to the chosen depth. Each node stores child links; leaves store a compact piece array. An OpenMP `target teams distribute parallel for` assigns different root subtrees to device workers. Each worker traverses its subtree with an explicit alpha-beta stack and evaluates visited leaves on the device. The best root move is reduced on the host. This returns the same fixed-depth score as the serial algorithm, subject to the same move and terminal rules.

This is a deliberately constrained GPU formulation: legal move generation and the full tree construction run on the CPU **before** offload. Consequently it may spend far more time and memory than serial alpha-beta, which generates only branches it needs. The total reported `seconds` includes CPU tree construction, transfer, and target search. `target_seconds` alone is not an end-to-end speedup. The original GPU version is limited by the number of root moves; neither GPU version spans multiple devices.

The CPU OpenMP/MPI task splits also lose pruning between independent tasks. Compare their node counts with serial alpha-beta before interpreting speedup. MPI currently uses round-robin task assignment, not dynamic load balancing.

Validation and current limits

`make test` compares serial alpha-beta, exhaustive minimax, OpenMP, and OpenMP target host-fallback scores on seven positions (including checkmate, stalemate, promotion, and en passant) at depths 1–3; it also checks CPU split-depth 2 and 3 where applicable. That checks the target algorithm's CPU execution path; it does **not** validate an actual GPU or MPI run on its own. Vanda MPI results and A40 offload results are recorded in `experiments/`, including `ran_on_device=1` and score comparisons with serial search.

Current implementation does not use transposition tables, iterative deepening, quiescence search, a sophisticated evaluation function, or dynamic MPI task scheduling. These are deliberate experimental boundaries rather than claims of a competitive engine. Multi-node scaling and GPU performance have been measured on Vanda, including a completed three-trial 720-core series.

AI usage

The project team developed the algorithms, implementations and experiments. OpenAI Codex assisted with preparing figures and drafting and editing the report. The report's AI-use declaration describes this assistance; the team checked its figures and conclusions against the archived Vanda results.